claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a1e34e8b51d44f2e4.jsonl`
- SHA-256: `18da2deaefb72608bd6893cdbbcfc856fa0c92f846b0a8ba37f960a2c7a39b66`
- Source modified: `2026-06-10T06:12:12+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:10:30.805Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("reliability") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"/api/process accepts an arbitrary server-side file path; /api/compile allows path traversal / absolute-path write via output_filename","severity":"high","area":"security / input validation","file":"backend/main.py:91-96 (video_path only os.path.exists), backend/main.py:200-204 + backend/pipeline/stitcher/compiler.py:75 (output_path=os.path.join(output_dir, output_filename))","evidence":"ProcessRequest.video_path (main.py:69) is an unconstrained str validated only by `os.path.exists` (main.py:93); the server then MD5s it, opens it with cv2, and feeds it to ffmpeg ? i.e. a client can point the service at any file the process can read. CompileRequest.output_filename (main.py:84) flows unchecked into `os.path.join(self.output_dir, output_filename)`; os.path.join discards output_dir when the second arg is absolute, and '..' segments escape it, so an ffmpeg mp4 can be written to an attacker-chosen location.","why_it_matters":"As a hosted paid API these are arbitrary-file-read (LFI/processing) and arbitrary-file-write primitives. DEFERRED_HARDENING_PLAN and PLATFORM_ARCHITECTURE mention auth/storage abstraction generically but never list input-path validation/traversal as a hardening item ? so this is a real gap in the deferral list, not a deliberately-deferred item.","why":"placeholder"}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:10:40.278Z)

```
1       import os
2       import sys
3       import argparse
4       import json
5       import logging
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10
11      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12      logging.basicConfig(
13          level=logging.INFO,
14          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15          datefmt='%H:%M:%S'
16      )
17      logger = logging.getLogger(__name__)
18      from fastapi import FastAPI, HTTPException
19      from fastapi.middleware.cors import CORSMiddleware
20      from pydantic import BaseModel
21      from typing import List, Dict, Any, Optional
22
23      from fastapi.staticfiles import StaticFiles
24      from fastapi.responses import HTMLResponse
25      from backend.pipeline.validators.base import registry
26      from backend.infrastructure.database import Database
27      from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
28      from backend.pipeline.stitcher.compiler import VideoCompiler
29      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
30      from backend.config import load_config as load_app_config, AppConfig  # new typed config
31      from backend.results import Result, Failure  # standardized error/result contract
32      from backend.reporting import emit_indexer_report
33
34      app = FastAPI(title="Sports Highlight Indexer API")
35
36      # Enable CORS for local web interface
37      app.add_middleware(
38          CORSMiddleware,
39          allow_origins=["*"],
40          allow_credentials=True,
41          allow_methods=["*"],
42          allow_headers=["*"],
43      )
44
45      # Serves static frontend files directly (now under api/ per approved structure)
46      os.makedirs("backend/api/static", exist_ok=True)
47      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
48
49      @app.get("/", response_class=HTMLResponse)
50      def serve_index():
51          index_path = "backend/api/static/index.html"
52          if os.path.exists(index_path):
53              with open(index_path, "r", encoding="utf-8") as f:
54                  return f.read()
55          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
56
57      # Global variables initialized at startup
58      db_instance: Optional[Database] = None
59      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
60
61      # Legacy thin wrapper for any remaining direct calls during transition.
62      # New code should import load_app_config / AppConfig from backend.config directly.
63      def load_config(config_path: str = "config.json") -> Dict[str, Any]:
```

```
64          cfg = load_app_config(config_path)
65          return cfg.model_dump(mode="json")
66
67      # --- API Models ---
68      class ProcessRequest(BaseModel):
69          video_path: str
70          player_pool: Optional[List[str]] = None
71          max_frames: Optional[int] = None
72          segmenter: Optional[str] = None
73
74      class QueryRequest(BaseModel):
75          video_id: str
76          server: Optional[str] = None
77          receiver: Optional[str] = None
78          duration_min: Optional[float] = None
79          event_type: Optional[str] = None
80
81      class CompileRequest(BaseModel):
82          video_id: str
83          segment_ids: List[int]
84          output_filename: str
85
86      # --- API Endpoints ---
87      @app.get("/api/config")
88      def get_config():
89          return global_config
90
91      @app.post("/api/process")
92      def process_video(req: ProcessRequest):
93          if not os.path.exists(req.video_path):
94              raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
95
96          video_id = compute_video_id(req.video_path)
97          was_reingest = db_instance.get_video(video_id) is not None
98
99          # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
100         logger.info(f"Running validations for {req.video_path}...")
101         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
102         failures = [r for r in val_results if not r.passed]
103
104         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
105         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
106         seg_name = req.segmenter or default_seg
107         segmenter_actual = None
108         segmentation_ran = False
109         response: Optional[Dict[str, Any]] = None
110         try:
111             if failures:
112                 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
113                 response = {
114                     "video_id": video_id,
115                     "status": "failed",
116                     "message": "Video failed validation checks.",
117                     "results": [r.dict() for r in val_results],
118                 }
119                 return response
120
121             # 2. Run segmenter & indexer
122             print(f"[API] Video passed checks. Segmenting and indexing...")
123             db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
```

```
val_results])

            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                available = list(segmenter_registry.list_available().keys())
                raise HTTPException(
                    status_code=400,
                    detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
                )

            print(f"[API] Using segmenter: {seg_name}")
            segmenter_actual = seg_name
            segmenter = segmenter_cls(db_instance, global_config)
            result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            segmentation_ran = True

            if isinstance(result, Failure):
                print(f"[API] Segmenter failed: {result.message}")
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.dict() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.dict() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
```

```
184     def compile_highlights(req: CompileRequest):
185         video = db_instance.get_video(req.video_id)
186         if not video:
187             raise HTTPException(status_code=404, detail="Indexed video record not found.")
188 
189         # Retrieve matching play segments from db
190         all_segments = db_instance.query_play_segments(req.video_id, {})
191         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
192 
193         if not matched_segments:
194             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
195 
196         # Extract start/end time pairs
197         time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]
198 
199         # Run compiler
200         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
201         compiler = VideoCompiler(output_dir)
202         try:
203             out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
204             return {"status": "success", "filepath": out_path}
205         except Exception as e:
206             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")
207 
208     # --- CLI Debug Utility & Orchestration ---
209     def run_cli_diagnose_clip(video_path: str, timestamp: float):
210         """CLI debugging utility to examine segment properties around a timestamp."""
211         print("=" * 60)
212         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
213         print("=" * 60)
214 
215         cfg = load_config()  # legacy wrapper still works, returns dict for now
216 
217         # 1. Validation check diagnostics
218         print("[DIAGNOSTIC] Running validation framework...")
219         results = registry.run_all(video_path, cfg)
220         for r in results:
221             status_symbol = "[PASS]" if r.passed else "[FAIL]"
222             print(f"  {status_symbol} {r.validator_name}: {r.message}")
223             if r.details:
224                 print(f"         Details: {r.details}")
225 
226         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
227         print("-" * 60)
228         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
229         import cv2
230         cap = cv2.VideoCapture(video_path)
231         if not cap.isOpened():
232             print("[FAIL] OpenCV could not open video.")
233             return
234 
235         fps = cap.get(cv2.CAP_PROP_FPS)
236         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
237 
238         start_frame = max(0, int((timestamp - 3.0) * fps))
239         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
240 
```

3. user (2026-06-10T06:10:40.966Z)

```
1       import os
```

```
2       import subprocess
3       import tempfile
4       import logging
5       from typing import List, Tuple, Union
6
7       logger = logging.getLogger(__name__)
8
9       class VideoCompiler:
10          def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
11              self.output_dir = output_dir
12              self.end_padding = end_padding
13              self.begin_padding = begin_padding
14              os.makedirs(output_dir, exist_ok=True)
15
16          @staticmethod
17          def _parse_time(val: Union[int, float, str]) -> float:
18              """Normalizes a timestamp value to float seconds.
19              Handles: float/int (pass-through), plain numeric strings ("12.4"),
20              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
21              This mirrors the parse_time logic used in gemini.py to ensure
22              the stitcher can always consume whatever format the indexer produced."""
23              if isinstance(val, (int, float)):
24                  return float(val)
25              if isinstance(val, str):
26                  val = val.strip()
27                  if ":" in val:
28                      # Handle MM:SS, HH:MM:SS, etc.
29                      parts = val.split(":")
30                      parts.reverse()
31                      total = 0.0
32                      for i, p in enumerate(parts):
33                          try:
34                              total += float(p) * (60 ** i)
35                          except ValueError:
36                              pass
37                      return total
38                  try:
39                      return float(val)
40                  except ValueError:
41                      logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
42                      return 0.0
43              logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
44              return 0.0
45
46          def _get_video_duration(self, video_path: str) -> float:
47              """Probes the video file to get its total duration in seconds.
48              Returns 0.0 if it cannot be determined."""
49              try:
50                  result = subprocess.run(
51                      [
52                          "ffprobe", "-v", "error",
53                          "-show_entries", "format=duration",
54                          "-of", "default=noprint_wrappers=1:nokey=1",
55                          video_path
56                      ],
57                      capture_output=True, text=True, check=True
58                  )
59                  return float(result.stdout.strip())
60              except Exception as e:
61                  logger.warning(f"Could not determine video duration via ffprobe: {e}")
62                  return 0.0
63
```

```
64          def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
65              """
66              Extracts selected segments from a raw video and stitches them together
67              into a seamless highlights file, applying end_padding to prevent abrupt endings.
68              """
69              if not segments:
70                  raise ValueError("No highlight segments provided to compile.")
71
72              if not os.path.exists(raw_video_path):
73                  raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
74
75              output_path = os.path.join(self.output_dir, output_filename)
76
77              # Probe video duration so we can detect out-of-range segments
78              video_duration = self._get_video_duration(raw_video_path)
79              if video_duration > 0:
80                  logger.info(f"Source video duration: {video_duration:.2f}s")
81              else:
82                  logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
83
84              # We will create temporary clips and stitch them using FFmpeg concat demuxer
85              temp_clips = []
86              skipped_segments = []
87
88              # Create a temp directory within output_dir
89              with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
90                  logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
91                  for idx, (raw_start, raw_end) in enumerate(segments):
92                      # Normalize timestamps to float seconds ? handles float, int,
93                      # plain numeric strings, and colon-separated formats like "MM:SS"
94                      start = self._parse_time(raw_start)
95                      end = self._parse_time(raw_end)
96                      segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
97
98                      # Validate the segment times
99                      if start < 0:
100                         logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
101                         start = 0.0
102
103                     if start >= end:
104                         logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
105                         skipped_segments.append((idx, start, end, "start >= end"))
106                         continue
107
108                     # Check if segment extends beyond video duration
109                     if video_duration > 0:
110                         if start >= video_duration:
111                             logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
112                                          f"This segment cannot be extracted ? the video ran
out. Skipping.")
113                             skipped_segments.append((idx, start, end, "start beyond video
end"))
114                             continue
115
116                         if end > video_duration:
117                             original_end = end
118                             end = video_duration
```

```
                            logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
                                           f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")

                    clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

                    # Precise sub-second trim using fast re-encoding to preserve stream
headers
                    padded_start = max(0.0, start - self.begin_padding)
                    padded_end = end + self.end_padding

                    # Clamp padded_end to video duration if known
                    if video_duration > 0 and padded_end > video_duration:
                        padded_end = video_duration
                        logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
                                    f"Clamping to {video_duration:.2f}s (end padding
partially applied).")

                    trim_duration = padded_end - padded_start
                    logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")

                    cmd = [
                        "ffmpeg", "-y",
                        "-ss", str(round(padded_start, 3)),
                        "-to", str(round(padded_end, 3)),
                        "-i", raw_video_path,
                        "-c:v", "libx264",
                        "-preset", "superfast",
                        "-crf", "22",
                        "-c:a", "aac",
                        "-b:a", "128k",
                        clip_path
                    ]

                    try:
                        result = subprocess.run(cmd, capture_output=True, check=True)
                        # Verify the output clip was actually created and has content
                        if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                            temp_clips.append(clip_path)
                        else:
                            logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
                            skipped_segments.append((idx, start, end, "empty output clip"))
                    except subprocess.CalledProcessError as e:
                        stderr_msg = e.stderr.decode(errors='replace').strip()
                        logger.error(f"{segment_label}: FFmpeg trim failed.")
                        print(f"  Command: {' '.join(cmd)}")
                        print(f"  FFmpeg stderr: {stderr_msg[-500:]}")  # Last 500 chars to
avoid flooding
                        skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
                        continue

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
                    logger.info("")

```

```
173                 if not temp_clips:
174                     raise RuntimeError(
175                         f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
176                         f"All {len(segments)} segments were skipped. "
177                         f"Check the errors above for details. "
178                         f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
179                     )
180 
181                 logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
182 
183                 # Create a list file for FFmpeg concat demuxer
184                 list_file_path = os.path.join(tmp_dir, "clips.txt")
185                 with open(list_file_path, "w") as f:
186                     for clip in temp_clips:
187                         # Escape single quotes in path
188                         safe_path = clip.replace("'", "'\\''")
189                         f.write(f"file '{safe_path}'\n")
190 
191                 logger.info("Stitching clips together...")
192                 # Stitch them together without re-encoding (since they were encoded with
identical settings)
193                 concat_cmd = [
194                     "ffmpeg", "-y",
195                     "-f", "concat",
196                     "-safe", "0",
197                     "-i", list_file_path,
198                     "-c", "copy",
199                     output_path
200                 ]
201 
202                 try:
203                     result = subprocess.run(concat_cmd, capture_output=True, check=True)
204 
205                     # Final verification
206                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
207                         raise RuntimeError(
208                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
209                             f"but output file is missing or empty: {output_path}"
210                         )
211 
212                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
213                     logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
214                     return output_path
215                 except subprocess.CalledProcessError as e:
216                     stderr_msg = e.stderr.decode(errors='replace').strip()
217                     raise RuntimeError(
218                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
219                         f"  Command: {' '.join(concat_cmd)}\n"
220                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
221                     )
222 
```

4. user (2026-06-10T06:11:08.961Z)

```
docs\EVALUATION.md
docs\TRACKNET_WSL_SETUP.md
docs\GOLDEN_SET_IMPLEMENTATION.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\archives\COMMERCIAL_READINESS_REVIEW.md
docs\archives\MONETIZATION_AUDIT.md
docs\archives\README.md
```

```
docs\DEFERRED_HARDENING_PLAN.md
docs\PMF_MARKET_RESEARCH.md
docs\ROADMAP.md
docs\archives\checkpoints\2026-06-eval-baseline\README.md
docs\archives\checkpoints\2026-06-logging-reporting\README.md
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\decisions\DECISIONS.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md
docs\archives\past_projects\README.md
docs\archives\research\AUDIO_E1_FINDINGS.md
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md
docs\archives\research\RALLY_DETECTION_RESEARCH.md
docs\COMMERCIALIZATION.md
docs\PLATFORM_ARCHITECTURE.md
docs\BACKLOG.md
docs\GOLDEN_SET_VENDORING.md
docs\README.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\archives\checkpoints\2026-06-strategy-foundation\README.md
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md
docs\archives\quality-iterations\2026-06-initial\README.md
docs\CODE_MAP.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md
docs\archives\quality-iterations\README.md
docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md
docs\NEXT_STEPS.md
```

5. user (2026-06-10T06:11:10.526Z)

```
docs\PLATFORM_ARCHITECTURE.md:102:Replace the bare `video_path: str` that flows through the
pipeline with a
docs\PLATFORM_ARCHITECTURE.md:464:| **P2** | **`StorageBackend` registry + BYO-storage** | new
ABC+registry; `local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted
per-tenant creds |
docs\CODE_MAP.md:25:  -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
docs\CODE_MAP.md:31:  -> segments = seg.process_video(video_id, video_path, player_pool?,
max_frames?)
docs\CODE_MAP.md:85:  - `main()` sets `global_config.output.output_dir = args.output_dir` (typed
field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir,
output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so
`--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
docs\CODE_MAP.md:107:  - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
docs\CODE_MAP.md:133:- `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in
run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1),
`::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`),
`::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
docs\CODE_MAP.md:155:[Omitted long matching line]
docs\CODE_MAP.md:176:[Omitted long matching line]
docs\TRACKNET_WSL_SETUP.md:93:python predict.py --video_path ~/clips/<clip>.mp4 \
```

6. assistant (2026-06-10T06:11:25.170Z)

The grep confirms the docs do NOT list path-traversal/input-path validation as a hardening item. Let me read the relevant doc sections to judge whether this is deliberately deferred and check the security posture / single-tenant framing.

7. user (2026-06-10T06:11:26.404Z)

```
90      else either exists (pipeline) or is net-new infra (identity, orchestration, serving). A
91      **model/training layer** ? the owned generative video-QA model (§5A) ? sits atop these,
```

```
92      fed by Storage (training corpus), Data (labels), and Compute (training + inference), and
93      is delivered through the existing **AI-provider registry with a paid fallback**.
94
95      ## 3. Pluggable abstractions (extend the existing registry pattern)
96
97      The codebase already proves the model: an `ABC` + a `Registry` singleton with
98      `register/get/list_available` (e.g. `SegmenterRegistry`). Add two more registries with
99      the same shape so they're plug-and-play and testable in isolation.
100
101     ### 3.1 `StorageBackend` registry
102     Replace the bare `video_path: str` that flows through the pipeline with a
103     **`StorageRef`** (a backend id + a key/URI), resolved by a backend.
104
105     ```python
106     class StorageBackend(ABC):
107         name: str
108         def open_read(self, ref: StorageRef) -> BinaryIO: ...      # stream large video
109         def download_to_local(self, ref: StorageRef, dst: str): ...  # for ffmpeg/cv2/WSL
110         def upload(self, local_path: str, ref: StorageRef): ...     # persist
highlight/artifacts
111         def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
112         def stat(self, ref: StorageRef) -> StorageStat: ...
113         def list(self, prefix: StorageRef) -> list[StorageRef]: ...
114     ```
115
116     - **Recommended implementation:** build the backend over **`fsspec`** (one matrix ?
117       local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) + Google Drive via `gdrivefs`,
118       Dropbox via `dropboxdrivefs`), reaching for **`smart_open`** as the robust large-MP4
119       byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as
```

8. user (2026-06-10T06:11:26.605Z)

```
1       # Deferred Hardening Plan ? Train/Eval Guardrails, Async Jobs, Packaging
2
3       **Status:** ? Design for review ? **no code yet**. Owner-requested follow-up to the
4       "config & identity consistency" tech-debt pass. Each item below becomes its own PR
5       only after this doc is approved.
6
7       **Why these three:** they were cataloged as out-of-scope during the consistency
8       pass because each is larger/design-heavy or needs the owner's machine. They are the
9       remaining high-value hygiene items not already done (config/structure/Result/logging/
10      reports) or in flight (detector abstraction, PR #49).
11
12      **Recommended sequence:** #7 Packaging (lowest risk, unblocks clean installs/CI) ?
13      #13 Train/eval guardrails (correctness of the data flywheel) ? #16 Async jobs
14      (largest, product-facing).
15
16      ---
17
18      ## 1. Packaging modernization (debt #7)
19
20      ### Problem
21      - No `pyproject.toml` on `master`. (PR #49's description lists packaging as "done",
22        but it isn't present ? reconcile with the owner.)
23      - The root `setup.py` is a **diagnostics/dependency-checker**, not a PEP 517 build
24        script ? its name collides with setuptools semantics and will confuse `pip`/`build`.
25      - Deps live only in `requirements.txt`; `obsreport` is a commented soft dep
26        (BACKLOG) and `supervision<0.30.0` is pinned (debt #14).
27
28      ### Approach
29      - Add `pyproject.toml` (PEP 621): project metadata + runtime deps mirroring
30        `requirements.txt`, with optional-dependency groups:
31        - `[project.optional-dependencies] dev` ? `pytest`, coverage tooling.
32        - `reporting` ? `obsreport` pinned to its git tag (`v0.1.3`, per BACKLOG).
33        - `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is **not**
```

```
    expressible in standard PEP 621 deps; document the install line or use a
    `[tool.uv]`/pip-conf approach rather than faking it).
- Console entry points: `indexer = backend.main:main` (so `indexer --server` works),
  optionally eval drivers.
- **Rename the diagnostic `setup.py`** (e.g. `scripts/doctor.py`) so it no longer
  shadows the build system; update README/`run_all_tests.py`/CI references.
- Choose a build backend (hatchling recommended) and confirm the `backend/` import
  root stays valid (keep current layout; no src/ move required).

### Risks
- `setup.py` rename touches docs and any muscle-memory invocations ? grep first.
- Torch CUDA wheels can't be pinned cleanly in PEP 621; must document the
  `--extra-index-url` install path (the current `setup.py` logic can move into docs).
- CI must switch to `pip install -e .[dev]`.

### Verification
- `pip install -e .[dev]` in a clean venv ? `run_all_tests.py` green.
- `indexer --help` resolves via the entry point.

---

## 2. Train/eval separation guardrails (debt #13)

### Problem
Separation is **by convention only**. `backend/eval/calibrate_local.py` and
`calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces*
that a video used to train the learned segmenter
(`backend/eval/training.py::build_training_set` ? `distill_local`) is absent from the
eval/regression set. Leakage would silently inflate the very metrics the product
story rests on (? in BACKLOG).

### Approach
- **Split unit = match, never clip** (per `GOLDEN_SET_PHASE1_VIDEOS.md`: split by
  match, hold ~? of each sport as eval).
- Add a `split: "train" | "eval"` field to the collector's `curate export`
  manifest (cross-repo contract, ADR-002 lineage) ? or a separate eval manifest.
- New `backend/eval/splits.py`:
  - `load_split(manifest) -> (train_ids, eval_ids)` keyed by **match id** (not just
    file MD5 ? re-encoded duplicates share a match but differ in
    `compute_video_id`, so group at match level).
  - `assert_disjoint(train_ids, eval_ids)` ? raise on any overlap.
- Wire the guard into the training entrypoint (`distill_local`/`training`) and into
  `run_regression.py` so the baseline is computed strictly on the eval split.

### Risks
- Requires a collector-side manifest change (coordinate the cross-repo contract).
- Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct
  but the numbers will be noisy until the golden set grows.
- MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.

### Verification
- Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
- Integration: a deliberately-leaked manifest fails the training entrypoint.

---

## 3. Async job model (debt #16)

### Problem
`POST /api/process` runs validate ? segment ? report **inline**, blocking the
request for the full duration of a long video. Unworkable as a hosted/product API.

### Approach (minimal first slice ? single self-hosted box)
- `POST /api/process` returns **202 + `job_id`**; add `GET /api/jobs/{job_id}`
  returning `{status, progress, result, reports}`.
```

```
99      - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100       (segmenters already use threads internally for AI handoff, so this fits); leave a
101       real queue (RQ/Celery) for later if multi-process is needed.
102     - Persist job state in the existing SQLite DB via the migration ladder
103       (`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
104       add a `jobs` table). The observability report becomes the job's artifact.
105     - Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.
106
107     ### Risks
108     - **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
109       `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
110       confirm it's safe under the executor.
111     - Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112     - In-process store is not horizontally scalable ? acceptable for the current
113       single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114
115     ### Verification
116     - Submit a job ? poll ? DONE with the same segments the sync path produced.
117     - Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.
118
119     ---
120
121     ## Out of this doc
122     - **Detector/WSL abstraction (#6):** in flight (PR #49).
123     - **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
124     - **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
125       need a GPU/WSL box and real video ? owner-machine work, tracked separately.
126
```

9. assistant (2026-06-10T06:11:32.183Z)

Now let me check the PLATFORM_ARCHITECTURE roadmap table and any auth/security section, plus whether CODE_MAP gotchas or any doc frames the current state as local single-tenant (which bears on severity).

10. user (2026-06-10T06:11:35.171Z)

```
6:**Purpose.** Map the path from today's **single-user, local** tool to a **multi-tenant
50:| Identity / tenancy | none | no users, orgs, auth, or isolation |
55:| API | `/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth | needs auth,
per-tenant scoping, async job + history endpoints |
70:   CONTROL      ?  Presentation / API  (auth, per-tenant)        ?
77:   PLANE        ?  (local | BYO-VM |    ?  ?  (per-tenant DB:    ?
122:  the tenant registers their own bucket/Drive; raw video **never leaves their account**.
123:- **Auth ? never persist users' long-lived keys.** For buckets, have the user create an
126:  servers**. For Drive, OAuth with the narrow **`drive.file`** scope; for academies, a
127:  **service account with domain-wide delegation**. Store any per-tenant creds encrypted.
140:- **`byo_worker`** ? the tenant runs a **pull-based worker** (a small agent) on their own
141:  VM/GPU. **Critical security property:** the worker *pulls* jobs scoped to its tenant
142:  from the control plane, pulls input from the tenant's **own** storage, runs the
162:  in a **strictly isolated, per-tenant, ephemeral sandbox** (nothing persists our side).
174:## 4. Multi-tenant data model & serving layer
192:- **`video_id` stays the content MD5** (dedup within a tenant), scoped by `tenant_id`.
205:| **Shared DB + `tenant_id`** (row-level, ideally Postgres **RLS**) | logical | low |
**Hosted SaaS, many small tenants** ? *recommended default* |
206:| Schema-per-tenant | medium | medium | tens?hundreds of mid tenants |
207:| **DB-per-tenant** (incl. **SQLite-per-tenant**) | strong/physical | higher at scale |
**self-hosted single-academy**, or high-isolation enterprise |
210:`tenant_id` + Row-Level Security**; keep **SQLite-per-tenant** as the **self-hosted /
219:that first, single-tenant, then generalize.
221:- **Job record** (`jobs` table): `id, tenant_id, video_id, type (index|compile|analyze),
224:- **Idempotency** keyed on `(tenant_id, video_id, pipeline_version)` ? reuse the
227:  for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline,
236:    Postgres-backed queue + worker** ? likely sufficient for the *first* single-tenant
296:inference jobs (even **BYO-compute** can serve inference on the tenant's GPU); the
```

```
430:  **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's bucket
436:  A small **per-user (or per-tenant) LoRA adapter, fine-tuned *only on that user's own
462:| **P0** | **Async job model, single-tenant** (DEFERRED #16) | `jobs` table via
`user_version` ladder; worker; 202+status endpoints |
463:| **P1** | **Identity + multi-tenant data model + upload + highlight persistence + history**
| auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS; `highlights` table;
repository layer; history API + UI |
464:| **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry;
`local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds
|
489:  decide whether it runs **locally/on-tenant only** (strong posture) vs our cloud. (EDPB
495:  tenant's own bucket/Drive and we process via signed URL or their compute, we minimize
505:  what merely passes through a tenant's bucket. **Exclude minors' video from the training
514:- **Right to erasure / retention:** per-tenant, per-athlete delete must cascade across DB +
521:   option ? it's the highest-complexity, highest-security item.
523:   SQLite-per-tenant if self-hosted-single-academy is the first GTM. (Abstract DB access
525:3. **Auth: build vs buy** (Auth0 / Clerk / Supabase Auth / Cognito).
528:5. **Gait processing location** ? local/on-tenant (privacy-max) vs cloud (capability-max).
529:6. **How far to push BYO-compute** given its security/ops complexity.
538:**Risks:** scope explosion (sequence strictly P0?P5); multi-tenant data-leak bugs (RLS +
539:tests); BYO-compute security; **biometric/minor liability** (get this right before any
553:**Multi-tenancy:** [Bytebase: multi-tenant DB patterns](https://www.bytebase.com/blog/multi-
tenant-database-architecture-patterns-explained/) · [Hunchbite: RLS vs schema-per-
tenant](https://hunchbite.com/guides/multi-tenant-saas-architecture) · [PlanetScale: tenancy in
Postgres](https://planetscale.com/blog/approaches-to-tenancy-in-postgres) · [DEV: DB-per-tenant
vs shared schema](https://dev.to/young_gao/multi-tenant-architecture-database-per-tenant-vs-
shared-schema-1n2e)
```

11. user (2026-06-10T06:11:36.818Z)

```
3:**Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted
video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
12:- `?` = gotcha / footgun
235:## 6. GOTCHAS (cross-cutting footguns)
274:| Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory
--labels --fps --frame-width`); trust `cv` recall not `fit` |
```

12. assistant (2026-06-10T06:11:42.802Z)

```
Let me check the CODE_MAP gotchas section (line 235+) to see if path handling is flagged as a
known footgun, and confirm there's no auth currently (line 50/55 confirm "none").
```

13. user (2026-06-10T06:11:43.805Z)

```
235     ## 6. GOTCHAS (cross-cutting footguns)
236     - **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
237     - **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config`/`.indexing` is `None` (the ASGI-import edge case).
238     - **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `'motion'`
getattr fallback noted above.
239     - **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
240     - **`output_dir`:** a real field on `OutputConfig` (default `"output"`); the CLI
`--output-dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the
same `global_config.output.output_dir` ? honored end-to-end.
241     - **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if
```

`{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze_video` also returns `[]` on real failure -> empty != "no rallies". The reporting `silent_provider_noop`/`ai_empty_vs_no_rallies` rules exist precisely to disambiguate this in the report.
242   - **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ? absent -> no-op.
243   - **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops firing. Add rules in spec.yaml, not code.
244   - **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`. The report's `fps_fallback_used` flag surfaces this.
245   - **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
246   - **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`. ? Don't import `wasb_infer` from Windows code (it imports torch + WASB repo modules).
247   - **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
248   - **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in `output/<db_name>` before any eval/regression.
249   - **Validator ordering coupling:** exec order is Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless `format_check` ran earlier (shared `context['ffprobe_meta']`). PTS 10.0s gap + scene-cut 0.6 correlation are NOT config-overridable.
250   - **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-deletes its segments. Use `clear_video_data` deliberately. `_get_connection` MUST re-issue `PRAGMA foreign_keys=ON` per connection.
251   - **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to P3/P4 must land in both. ? subtle drift: wasb's shot_count `int(None)`-via-TypeError fallback vs tracknet's `is None` check.
252   - **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs `backend.annotations` (provider registry). regression.py imports the latter.
253   - **`calibrate_wasb.{load_golden_gts, make_predict_fn, BASELINE}` is a de-facto shared API** for calibrate_local, export_wasb_segments, gemini_refine, e1_probe ? changing its golden-CSV schema or `BASELINE` ripples widely. `gemini_refine.merge_overlaps` reused by `distill_local`; `harness.get_predictions` reused by `regression.regress`.
254   - **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
255   - **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
256   - **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.
257   - **Audio NOT in the live pipeline:** `pipeline/audio/hit_detector` + `eval/audio/e1_probe` are a diagnostic probe (ADR-007); recall 100% but discrimination ~2.2× -> not adopted for fusion.
258
259   ---
260
261   ## 7. RAMP PATHS (to do X -> open Y::symbol)
262 | Task | Open |
263 |---|---|
264 | Change a config default / add a config field | `@backend/config/models.py` (single source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
265 | Read config in new code | `from backend.config import load_config, AppConfig` (`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
266 | Tune candidate windowing (velocity/merge/min-dur) | `@backend/pipeline/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration` |
267 | Add a segmenter | subclass `@backend/pipeline/segmenters/base.py::BasePlaySegmenter`; `segmenter_registry.register(...)` at module bottom; import in `@backend/pipeline/segmenters/__init__.py`; if AI-based add to

`@backend/reporting/harvest.py::_AI_SEGMENTERS` |
268 | Change stitching padding | `config.json stitching.{begin_padding,end_padding}`; impl `@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror `@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
269 | Add a sport | subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file |
270 | Add an AI provider | subclass `@backend/providers/base.py::AIVideoProvider`; `provider_registry.register('name', Cls)` |
271 | Add an annotation/GT tier | subclass `@backend/annotations/base.py::AnnotationProvider`; register + import in `@backend/annotations/__init__.py` |
272 | Add / change a report footgun rule | `@backend/reporting/spec.yaml::rules` (declarative; ? `when.path` must match harvest's stage/metric/detail names); facts in `@backend/reporting/harvest.py` |
273 | Tweak what the report records | `@backend/reporting/harvest.py::record_run` + stage builders; emit point is `@backend/main.py` `finally:` -> `@backend/reporting/__init__.py::emit_indexer_report` |
274 | Run calibration (WASB thresholds) | `@backend/eval/calibrate_wasb.py::main` (`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit` |

14. user (2026-06-10T06:11:44.714Z)

460 | Phase | Deliverable | Build / reuse |
461 |---|---|---|
462 | **P0** | **Async job model, single-tenant** (DEFERRED #16) | `jobs` table via `user_version` ladder; worker; 202+status endpoints |
463 | **P1** | **Identity + multi-tenant data model + upload + highlight persistence + history** | auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS; `highlights` table; repository layer; history API + UI |
464 | **P2** | **`StorageBackend` registry + BYO-storage** | new ABC+registry; `local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds |
465 | **P3** | **`ComputeBackend` registry + BYO-compute** | `local`/`hosted`/`byo_worker`; pull-based worker agent; control/data-plane split |
466 | **P4** | **Rich rally/shot semantic metadata & tagging** *(the next value stage after solid rally detection + highlights)* | shot type (smash / backhand drive / net), rally **outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ? query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden set via the `annotation` registry + `candidate_segments` |
467 | **P5** | **Owned generative video-QA model** (§5A) | global flywheel **+ per-user adapters**; P4 metadata is training signal ? fine-tune an open VLM (Qwen-VL) via LoRA ? serve as `owned_vlm` **provider with Gemini paid fallback**; extend the eval harness to QA quality |
468 | **P6** *(visionary)* | **Gait / biomechanics under a supervising coach** | deliberately **back-burner**; technique feedback; highest Art. 9 bar ? gate behind explicit biometric + parental consent (§7) |
469
470   P0?P1 deliver the headline ask (per-user upload + stored highlights + history). P2?P3
471   deliver the "plug-and-play storage/compute" ask. **P4 (rich semantic metadata) is the

15. assistant (2026-06-10T06:12:00.188Z)

I have enough to render a verdict. Let me synthesize.

Code claims ? all verified accurate:
- `ProcessRequest.video_path: str` (main.py:69), validated only by `os.path.exists(req.video_path)` (main.py:93), then MD5'd (main.py:96), opened by validators/cv2/ffmpeg (main.py:101, 136). Confirmed: arbitrary server-readable path.
- `CompileRequest.output_filename: str` (main.py:84) ? `compiler.compile_highlights(..., req.output_filename)` (main.py:203) ? `output_path = os.path.join(self.output_dir, output_filename)` (compiler.py:75), then written by ffmpeg (compiler.py:199, 203). `os.path.join` does discard `output_dir` on an absolute second arg, and `..` escapes ? standard Python behavior, accurately described. No sanitization anywhere in the path.

The auditor's "new gap" claim ? partially wrong:
The deferral docs DO anticipate the `video_path` issue, just not under a "path traversal" label.

PLATFORM_ARCHITECTURE.md:102 and 464 (P2) explicitly call out replacing `video_path: str` with a `StorageRef` (backend id + key resolved by a backend), which structurally removes the arbitrary-path primitive. CODE_MAP.md:50/55 document that the API has **no auth / no tenant isolation** today as a known state, and the whole doc frames this as "today's single-user, local tool" (CODE_MAP.md:3,6). So "never mentioned" is an overstatement: the input-path-as-`StorageRef` redesign is on the roadmap, though `output_filename` traversal specifically is not separately itemized.

Severity ? materially overstated for current state. The system is, by its own docs, a single-user, local, unauthenticated, self-hosted tool (CODE_MAP.md:3,6,50,55; CLAUDE.md "single-tenant"). The operator who runs the server is the same person who owns the files ? pointing it at a local file or choosing an output path is the intended workflow, not a privilege boundary crossing. The "hosted paid API" framing is a FUTURE state (P1 adds auth, P2 replaces `video_path`), and platform implementation has not started. There is no current trust boundary for these to cross.

The finding is technically true as code description but the "high"-severity arbitrary-file-read/write framing assumes a hosted multi-tenant deployment that does not exist, and the "real gap never mentioned in the deferral list" claim is partly refuted by the documented `video_path`?`StorageRef` redesign and the documented no-auth/no-isolation state.

16. user (2026-06-10T06:12:04.925Z)

Structured output provided successfully

17. assistant (2026-06-10T06:12:12.150Z)

I've completed the adversarial verification. Verdict: **isReal=false, high confidence.**

The code-level technical claims are accurate (verified at backend/main.py:69/93/84/203 and backend/pipeline/stitcher/compiler.py:75), but the finding fails on two grounds:

1. **Severity is materially overstated.** The system is, per its own docs, a single-user, local, unauthenticated, self-hosted tool (docs/CODE_MAP.md:3,6,50,55; CLAUDE.md "single-tenant"). The operator running the server owns the files; supplying a local `video_path` and choosing an output path is the intended workflow, not a privilege-boundary crossing. The "hosted paid API ? arbitrary file read/write" framing assumes a multi-tenant deployment that does not exist and has not been started.

2. **The "real gap never mentioned in the deferral list" claim is partly refuted.** docs/PLATFORM_ARCHITECTURE.md:102 and the P2 roadmap row (line 464) explicitly call for replacing the bare `video_path: str` with a resolved `StorageRef`, which removes the arbitrary-path primitive; docs/CODE_MAP.md:50/55 document the current no-auth / no-tenant-isolation state as known. Only the `output_filename` traversal specifically is unenumerated ? not enough to make this a high-severity gap in the present design.